

Intermediate (build real memory) — Hints

[Back to the exercise](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Only 2 hints — work through them in order, and don't jump ahead until you've genuinely tried. Each hint has 3 depth levels: **Basic** (the plain idea), **Intermediate** (proper Python), **Advanced** (what breaks once more than one conversation exists at once). Read Basic first even if you already know Python — it's the fastest way to spot exactly what each deeper level adds.

- [Hint 1 — The idea, and the classic bug](#)
- [Hint 2 — The plan, and almost the whole thing](#)

[Back to the exercise](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Hint 1 — The idea, and the classic bug

Basic Version

There's no built-in "memory" to turn on. You build memory yourself, by keeping a growing list of every message so far, and sending the *whole* list again every time you call the model.

Start with a list that has just your system message in it. Each time around your loop: add the user's new message to the list, send the whole list, get a reply, add the reply to the list too.

Watch out for this exact bug: forgetting to append the assistant's reply back into the history list. If you skip that step, the model never sees its own previous answers — it'll look like it "forgot" everything within a couple of turns, and it's the single most common mistake here.

[Back to the exercise](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Intermediate Version

The whole exercise is one idea: a Python list, mutated in place, that grows by exactly two entries per turn — one `{"role": "user", ...}` and one `{"role": "assistant", ...}`. Every single call sends this entire list, not just the newest message; the model has no server-side memory of its own (this is the exact "No memory" idea from Doc03).

The response object doesn't give you back a ready-made message dict — you build one yourself from the reply text: `{"role": "assistant", "content": response.choices[0].message.content}`. Append this to `history`, not just print it, or the next turn's call won't include it.

Also decide how the loop ends — a sentinel input like typing `"quit"` is the simplest approach: check for it right after reading input, before appending anything, so the exit command itself never becomes part of the conversation history sent to the model.

[Back to the exercise](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Advanced Version

This exercise's `history` is one Python list, built inside one script, for one conversation at a time — which is fine here, but ask: **what happens the moment two conversations need to exist at once?** A web server handling two different users, or a test file running alongside your main script, can't both mutate one shared, global `history` list — one user's messages would leak into another user's conversation, silently, which is a real, embarrassing bug class in chat products, not a hypothetical one.

The other real question: `history` grows forever, by design, in this exercise. Nothing here stops it from eventually hitting the context-limit error you'll cause on purpose in this document's Edge cases exercise. A real chat feature needs *some* answer to "what happens when history gets too long" — even if the answer for now is just "not yet, but here's where that logic would go."

The fix for the first problem is to stop using a bare list at module or global scope, and instead bundle `history` and the behavior that mutates it together — in a small class, so each conversation gets its own instance instead of sharing one list:

```
class ChatSession:
    def __init__(self, client, system_prompt):
        self.client = client
        self.history = [{"role": "system", "content": system_prompt}]

    def send(self, user_input):
        # your turn: append user, call the API, append + return the reply
        ...
```

Sketch `send()`'s body yourself before checking Hint 2.

Difference between Basic, Intermediate, and Advanced: Basic and Intermediate both build memory correctly for one conversation, one script, one `history` list — Intermediate just names the exact pieces (the message-dict shape, the exit condition) more precisely. Advanced asks what breaks once that assumption — "there's only ever one conversation" — stops being true, which happens the moment this code is reused inside anything bigger than a single terminal script, and wraps `history` inside a class so each conversation owns its own list instead of all of them fighting over one shared global.

[Back to the exercise](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Hint 2 — The plan, and almost the whole thing

Basic Version

```
history = [system message]

loop forever:
    read user_input
    if user_input is "quit": stop the loop

    add {user, user_input} to history
    send history to the model, get reply
    add {assistant, reply} to history
    print reply
```

Here's almost the whole thing — just try running it and reading it line by line:

```
history = [
    {"role": "system", "content": "You are a helpful assistant."}
]

while True:
    user_input = input("You: ")
    if user_input == "quit":
        break

    history.append({"role": "user", "content": user_input})

    response = client.chat.completions.create(
        model="gpt-4o-mini",
        messages=history,
    )
    reply = response.choices[0].message.content

    history.append({"role": "assistant", "content": reply})
    print("Assistant:", reply)
```

Expected behavior: tell it your favorite color in turn 1, ask "what's my favorite color?" in turn 2 — if `history` is being built and sent correctly, turn 2's answer references it correctly.

[Back to the exercise](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Intermediate Version

```
function get_reply(history) -> str:
    call the API with messages=history
    return the reply text

function run_chat():
    history = [system message]
    loop:
        read user_input; if "quit": stop
        history.append(user message)
        reply = get_reply(history)
        history.append(assistant message)
        print reply
```

```
def get_reply(history: list) -> str:
    response = client.chat.completions.create(
        model="gpt-4o-mini",
        messages=history,
    )
    return response.choices[0].message.content

def run_chat() -> None:
    history = [
        {"role": "system", "content": "You are a helpful assistant."}
    ]

    while True:
        user_input = input("You: ")
        if user_input == "quit":
            break

        history.append({"role": "user", "content": user_input})
        reply = get_reply(history)
        history.append({"role": "assistant", "content": reply})

        print("Assistant:", reply)
```

Trace through 2-3 turns by hand and confirm `history` has the length you expect after each one, then write the `client` `/import` lines yourself before checking Hint 3's Advanced level.

[Back to the exercise](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Advanced Version

```
class ChatSession:
    __init__(client, system_prompt): store client, start self.history with system

    method send(user_input) -> str:
        append user message to self.history
        reply = call the API with self.history
        append assistant message to self.history
        return reply

usage:
    session_a = ChatSession(client, "...")    # one user's conversation
    session_b = ChatSession(client, "...")    # a completely separate one
    # session_a.history and session_b.history never touch each other
```

Turning that into real code — fill in the missing piece yourself:

```
from openai import OpenAI

class ChatSession:
    def __init__(self, client: OpenAI, system_prompt: str) -> None:
        self.client = client
        self.history: list[dict] = [{"role": "system", "content": system_prompt}]

    def send(self, user_input: str) -> str:
        self.history.append({"role": "user", "content": user_input})
        # your turn: call the API with self.history, append the reply
        # back into self.history, and return the reply text
        ...
```

Expected output if you build two independent `ChatSession` s and tell one your name, then ask the other one your name: the second session has no idea — proving the two histories really are separate, not one shared list in disguise.

Fill in `send()` yourself, then compare all 3 of your finished versions against the [Solution](#).

Difference between Basic, Intermediate, and Advanced: Basic and Intermediate both build one correct, growing `history` list for one conversation. Advanced wraps the exact same append-call-append logic inside a class, so `history` belongs to a `ChatSession` instance instead of living

loose in the script — the moment you need a second, independent conversation (which this document's Build Task needs, and any real chat product needs immediately), this is the difference between "just create another `ChatSession` " and "carefully untangle one shared list that two users have been silently sharing."

[Back to the exercise](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Full solution: [Show me the solution](#)